

1 Advantage Actor-Critic para Pong Vetorial

1.1 Visão Geral

Este projeto implementa um agente de Reinforcement Learning baseado no algoritmo **Advantage Actor-Critic (A2C)** para um ambiente Pong customizado.

Diferentemente do ambiente Atari tradicional, o agente não recebe imagens como entrada. O estado do ambiente é representado diretamente por variáveis geométricas e cinemáticas do jogo.

A implementação utiliza:

- Single Worker (1 ambiente)
- Política Estocástica
- Advantage Actor-Critic (A2C)
- Retorno n-Step
- Rede Neural MLP Compartilhada
- Otimizador Adam

2 Representação do Estado

O estado observado pelo agente é definido como:

$$s_t = \begin{bmatrix} ball_x \\ ball_y \\ ball_{vx} \\ ball_{vy} \\ agent_y \\ opponent_y \\ (ball_y - agent_y) \end{bmatrix} \quad (1)$$

onde:

- $ball_x$: posição horizontal da bola;
- $ball_y$: posição vertical da bola;
- $ball_{vx}$: velocidade horizontal da bola;
- $ball_{vy}$: velocidade vertical da bola;
- $agent_y$: posição da raquete do agente;
- $opponent_y$: posição da raquete adversária;
- $(ball_y - agent_y)$: distância vertical entre a bola e a raquete do agente.

Todos os estados são normalizados para o intervalo $[0, 1]$ antes de serem fornecidos à rede neural.

3 Espaço de Ações

O agente possui três ações discretas, conforme apresentado na Tabela 1:

Ação	Descrição
0	Permanecer parado
1	Mover para cima
2	Mover para baixo

Table 1: Espaço de ações do agente.

4 Função de Recompensa

A função de recompensa utilizada é definida pela Equação 2:

$$r_t = \begin{cases} +1, & \text{se o agente marcar um ponto} \\ -1, & \text{se o adversário marcar um ponto} \\ 0, & \text{caso contrário} \end{cases} \quad (2)$$

5 Arquitetura da Rede Neural

A arquitetura utiliza uma rede compartilhada para extração de características.

$$h_t = f_{\theta}(s_t) \quad (3)$$

onde:

- s_t representa o estado atual;
- h_t representa a representação latente produzida pela rede compartilhada;
- θ representa os parâmetros compartilhados.

A representação latente é utilizada simultaneamente pelo ator e pelo crítico.

5.1 Ator

A cabeça do ator produz um vetor de logits:

$$z_t = W_a h_t + b_a \quad (4)$$

Os logits são transformados em probabilidades através da função Softmax:

$$\pi(a|s_t) = \frac{\exp(z_a)}{\sum_i \exp(z_i)} \quad (5)$$

A ação executada é então amostrada segundo:

$$a_t \sim \pi(a|s_t) \quad (6)$$

onde a ação a_t pertence ao conjunto discreto:

$$a_t \in \{stay, up, down\} \quad (7)$$

No ambiente implementado, essas ações são codificadas numericamente como:

$$\mathcal{A} = \{0, 1, 2\} \quad (8)$$

com:

- 0: permanecer parado (*stay*);
- 1: mover para cima (*up*);
- 2: mover para baixo (*down*).

5.2 Crítico

A cabeça do crítico estima o valor esperado do estado:

$$V(s_t) = W_c h_t + b_c \quad (9)$$

Este valor representa a recompensa futura esperada ao partir do estado atual.

6 Retorno n-Step

Após a coleta de n transições, calcula-se o retorno n-Step:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots + \gamma^n V(s_{t+n}) \quad (10)$$

onde:

- γ é o fator de desconto;
- $V(s_{t+n})$ é a estimativa fornecida pelo crítico.

7 Função Vantagem

A função vantagem é definida como:

$$A(s_t, a_t) = G_t - V(s_t) \quad (11)$$

A vantagem indica quão melhor ou pior uma ação foi em comparação à expectativa do crítico.

8 Atualização do Ator

O ator é atualizado através de Policy Gradient utilizando a vantagem calculada.

A função objetivo é dada por:

$$\nabla J(\theta) = A(s_t, a_t) \nabla_{\theta} \log \pi(a_t | s_t) \quad (12)$$

A função de perda utilizada durante o treinamento é:

$$L_{actor} = -\log \pi(a_t | s_t) A(s_t, a_t) \quad (13)$$

A regra de atualização dos parâmetros do ator é:

$$\theta \leftarrow \theta + \alpha A(s_t, a_t) \nabla_{\theta} \log \pi(a_t | s_t) \quad (14)$$

onde α representa a taxa de aprendizado.

9 Atualização do Crítico

O crítico é treinado para aproximar o retorno observado:

$$V(s_t) \approx G_t \quad (15)$$

A função de perda utilizada é:

$$L_{critic} = (G_t - V(s_t))^2 \quad (16)$$

A atualização dos parâmetros do crítico é realizada através de gradiente descendente:

$$\phi \leftarrow \phi - \alpha \nabla_{\phi} (G_t - V_{\phi}(s_t))^2 \quad (17)$$

onde ϕ representa os parâmetros do crítico.

10 Regularização por Entropia

Para incentivar exploração durante o treinamento é utilizado um termo de entropia:

$$H(\pi) = - \sum_a \pi(a|s) \log \pi(a|s) \quad (18)$$

Políticas com maior entropia apresentam maior exploração das ações disponíveis.

11 Função de Perda Total

A função de perda completa utilizada pelo algoritmo é:

$$L = L_{actor} + c_v L_{critic} - c_e H(\pi) \quad (19)$$

onde:

- c_v é o peso da perda do crítico;
- c_e é o peso do termo de entropia.

12 Fluxo Geral de Treinamento

O treinamento segue os seguintes passos:

1. Observar o estado atual s_t ;
2. Produzir uma representação latente h_t através da rede compartilhada;
3. Calcular a política $\pi(a|s_t)$ através do ator;
4. Calcular o valor $V(s_t)$ através do crítico;
5. Amostrar uma ação a_t ;
6. Executar a ação no ambiente;
7. Coletar recompensas por n passos;
8. Calcular o retorno G_t ;
9. Calcular a vantagem $A(s_t, a_t)$;
10. Atualizar ator e crítico;
11. Repetir até o término do treinamento.

13 Discussão da Arquitetura Utilizada

A Figura 1 apresenta o fluxo completo do algoritmo implementado e será utilizada como referência para a discussão da arquitetura.

O estado vetorial do ambiente é inicialmente processado por uma rede neural compartilhada responsável pela extração de características relevantes do jogo.

A utilização de uma rede compartilhada reduz a quantidade total de parâmetros e permite que ator e crítico aprendam a partir da mesma representação latente do estado.

O ator utiliza essa representação para construir uma distribuição de probabilidades sobre as ações disponíveis.

Diferentemente de uma política determinística, a ação não é escolhida diretamente pela maior probabilidade.

A ação é amostrada segundo:

$$a_t \sim \pi(a|s_t) \quad (20)$$

Esse comportamento estocástico permite exploração durante o treinamento e evita convergência prematura para estratégias inadequadas.

O crítico utiliza a mesma representação latente para estimar:

$$V(s_t) \quad (21)$$

Essa estimativa é utilizada para calcular a vantagem apresentada na Equação 11 e reduzir a variância das atualizações da política.

Como o estado do ambiente já fornece diretamente informações geométricas relevantes da bola e das raquetes, uma arquitetura MLP simples mostrou-se suficiente para o problema estudado.

Dessa forma, não foi necessária a utilização de redes convolucionais, normalmente empregadas em ambientes baseados em imagens.

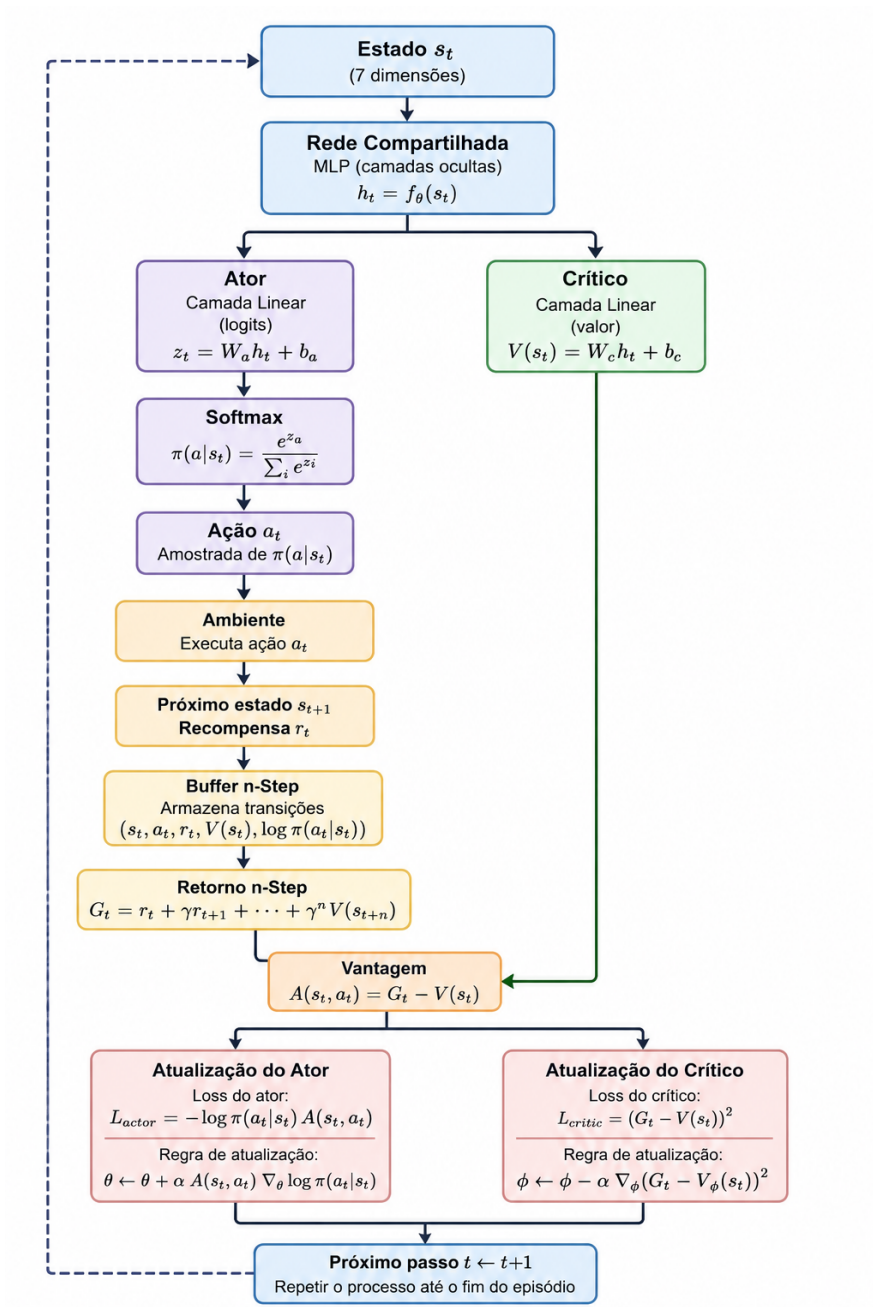


Figure 1: Arquitetura e fluxo de treinamento do algoritmo A2C.

14 Configuração Experimental

O treinamento foi realizado utilizando os hiperparâmetros apresentados na Tabela 2, que resume as principais escolhas utilizadas no experimento.

Parâmetro	Valor
EPISODES	10000
N_STEPS	64
GAMMA	0.99
LR	3×10^{-4}
HIDDEN_SIZE	256
ENTROPY_COEF	0.001
VALUE_COEF	0.5
MAX_GRAD_NORM	0.5
USE_GPU	False
NORMALIZE_ADVANTAGE	False

Table 2: Hiperparâmetros utilizados durante o treinamento.

O valor de $N_STEPS = 64$ foi escolhido para aumentar a propagação temporal da recompensa em um problema caracterizado por recompensas esparsas.

O fator de desconto $\gamma = 0.99$ permite que recompensas futuras continuem influenciando significativamente o processo de decisão.

A taxa de aprendizado utilizada foi de 3×10^{-4} , buscando um equilíbrio entre velocidade de aprendizado e estabilidade numérica.

O coeficiente de entropia foi mantido em 0.001 para incentivar exploração durante todo o treinamento.

O coeficiente do crítico foi definido como 0.5 para balancear a contribuição da função de valor na perda total.

Por fim, utilizou-se limitação de gradiente através de:

$$\|g\| \leq 0.5 \quad (22)$$

reduzindo possíveis instabilidades causadas por atualizações excessivamente grandes.

15 Resultados Experimentais

A evolução da recompensa por episódio e da média móvel é apresentada na Figura 2. Essa figura permite avaliar visualmente a transição entre a fase inicial de exploração e a fase posterior de desempenho positivo do agente.

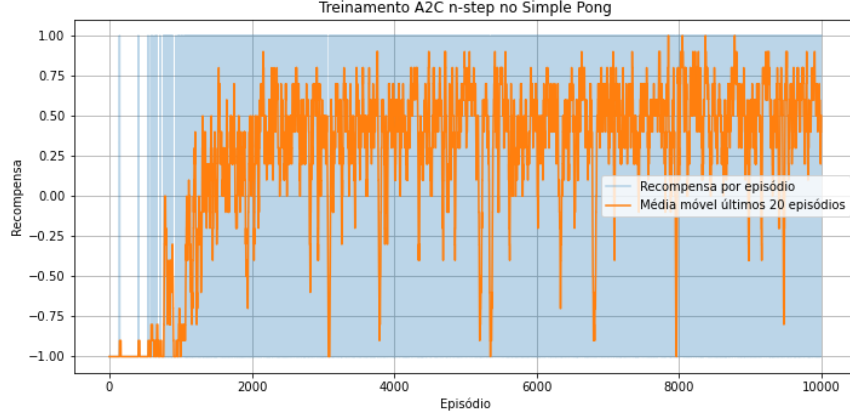


Figure 2: Evolução da recompensa por episódio e da média móvel dos últimos 20 episódios durante o treinamento do agente A2C.

16 Discussão dos Resultados

Observa-se, na Figura 2, uma fase inicial de exploração caracterizada por recompensas predominantemente negativas.

Após aproximadamente os primeiros milhares de episódios, a média móvel passa a apresentar tendência positiva, indicando aprendizado da política.

Apesar da melhora observada, a recompensa por episódio continua apresentando elevada variabilidade.

Esse comportamento observado na Figura 2 é esperado devido a três fatores principais.

Primeiramente, o ambiente utiliza uma função de recompensa esparsa, definida anteriormente pela Equação 2.

A maior parte dos estados recebe recompensa nula, enquanto recompensas positivas ou negativas são observadas apenas ao final das jogadas.

Além disso, as ações são escolhidas de forma estocástica através da distribuição:

$$a_t \sim \pi(a|s_t) \quad (23)$$

Consequentemente, mesmo após o aprendizado de uma boa política, ações menos prováveis ainda podem ser selecionadas ocasionalmente.

Outro fator relevante é a utilização da regularização por entropia.

O termo:

$$-c_e H(\pi) \quad (24)$$

presente na função de perda total da Equação 19 incentiva exploração contínua e evita convergência prematura para estratégias subótimas.

O retorno n-Step também desempenha papel importante, conforme definido pela Equação 10.

Como a recompensa é esparsa, utilizar:

$$N_STEPS = 64 \quad (25)$$

permite que informações de vitória ou derrota sejam propagadas para estados observados muitos passos antes da ocorrência da recompensa.

Por fim, a ausência de normalização da vantagem mostrou-se adequada para este problema.

Em cenários com recompensas esparsas, a normalização local pode reduzir excessivamente a magnitude do sinal de aprendizado.

Manter a vantagem em sua escala original preserva melhor as diferenças entre trajetórias bem-sucedidas e malsucedidas.

O código completo utilizado para gerar os resultados discutidos na Figura 2 é apresentado no Apêndice.

A Código Completo do Treinamento

```
% # train_a2c_simple_pong.py

import time
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.distributions import Categorical
import matplotlib.pyplot as plt
import sys

PROJECT_DIR = r"C:\Kegle_Jojo\Reinforcement-Learning-Examples\Pong_A2C"
sys.path.insert(0, PROJECT_DIR)

from single_pong_env import SimplePongEnv

EPISODES = 10000
N_STEPS = 64
GAMMA = 0.99
LR = 3e-4

HIDDEN_SIZE = 256

ENTROPY_COEF = 0.001
VALUE_COEF = 0.5
MAX_GRAD_NORM = 0.5

USE_GPU = False
```

```

NORMALIZE_ADVANTAGE = False

PLOT_PATH = "a2c_simple_pong_rewards.png"

class ActorCriticMLP(nn.Module):
    def __init__(self, obs_dim, n_actions, hidden_size=256):
        super().__init__()

        self.shared = nn.Sequential(
            nn.Linear(obs_dim, hidden_size),
            nn.Tanh(),
            nn.Linear(hidden_size, hidden_size),
            nn.Tanh()
        )

        self.actor = nn.Linear(hidden_size, n_actions)
        self.critic = nn.Linear(hidden_size, 1)

    def forward(self, x):
        features = self.shared(x)
        logits = self.actor(features)
        value = self.critic(features)
        return logits, value

def build_obs_normalizer(env):
    obs_low = env.observation_space.low.astype(np.float32)
    obs_high = env.observation_space.high.astype(np.float32)

    if np.any(np.isinf(obs_low)) or np.any(np.isinf(obs_high)):
        raise ValueError("observation_space possui limites infinitos.")

    if np.any(obs_high <= obs_low):
        raise ValueError("observation_space possui limites inválidos.")

    def normalize_obs(obs):
        obs = np.asarray(obs, dtype=np.float32)

        obs_norm = 2.0 * (obs - obs_low) / (obs_high - obs_low + 1e-8) -
            ↪ 1.0

        return np.clip(obs_norm, -1.0, 1.0).astype(np.float32)

    return normalize_obs

def compute_n_step_returns(rewards, dones, last_value, gamma):
    returns = []
    R = last_value

```

```

for reward, done in zip(reversed(rewards), reversed(dones)):
    if done:
        R = 0.0

    R = reward + gamma * R
    returns.insert(0, R)

return returns

def should_render_episode(episode, total_episodes):
    return episode <= 5 or episode > total_episodes - 5

def train():
    start_time = time.time()

    device = torch.device(
        "cuda" if USE_GPU and torch.cuda.is_available() else "cpu"
    )

    print(f"Usando dispositivo: {device}")

    env = SimplePongEnv(render_mode=None)

    normalize_obs = build_obs_normalizer(env)

    obs_dim = env.observation_space.shape[0]
    n_actions = env.action_space.n

    model = ActorCriticMLP(
        obs_dim=obs_dim,
        n_actions=n_actions,
        hidden_size=HIDDEN_SIZE
    ).to(device)

    optimizer = optim.Adam(model.parameters(), lr=LR)

    episode_rewards = []

    for episode in range(1, EPISODES + 1):
        render_this_episode = should_render_episode(episode, EPISODES)
        env.render_mode = "human" if render_this_episode else None

        obs, info = env.reset()
        obs = normalize_obs(obs)

        done = False
        episode_reward = 0

        log_probs = []

```

```

values = []
rewards = []
dones = []
entropies = []

while not done:
    obs_tensor = torch.tensor(
        obs,
        dtype=torch.float32,
        device=device
    ).unsqueeze(0)

    logits, value = model(obs_tensor)
    dist = Categorical(logits=logits)

    action = dist.sample()
    log_prob = dist.log_prob(action)
    entropy = dist.entropy()

    next_obs, reward, terminated, truncated, info =
    ↪ env.step(action.item())
    next_obs = normalize_obs(next_obs)

    done = terminated or truncated

    log_probs.append(log_prob.squeeze())
    values.append(value.squeeze())
    rewards.append(float(reward))
    dones.append(done)
    entropies.append(entropy.squeeze())

    obs = next_obs
    episode_reward += reward

    if len(rewards) >= N_STEPS or done:
        with torch.no_grad():
            if done:
                last_value = 0.0
            else:
                next_obs_tensor = torch.tensor(
                    obs,
                    dtype=torch.float32,
                    device=device
                ).unsqueeze(0)

                _, next_value = model(next_obs_tensor)
                last_value = next_value.item()

        returns = compute_n_step_returns(
            rewards=rewards,
            dones=dones,

```

```

        last_value=last_value,
        gamma=GAMMA
    )

    returns = torch.tensor(
        returns,
        dtype=torch.float32,
        device=device
    )

    values_tensor = torch.stack(values)
    log_probs_tensor = torch.stack(log_probs)
    entropies_tensor = torch.stack(entropies)

    advantages = returns - values_tensor.detach()

    if NORMALIZE_ADVANTAGE and len(advantages) > 1:
        advantages = (
            advantages - advantages.mean()
        ) / (advantages.std() + 1e-8)

    actor_loss = -(log_probs_tensor * advantages).mean()
    critic_loss = (returns - values_tensor).pow(2).mean()
    entropy_loss = entropies_tensor.mean()

    loss = (
        actor_loss
        + VALUE_COEF * critic_loss
        - ENTROPY_COEF * entropy_loss
    )

    optimizer.zero_grad()
    loss.backward()

    if MAX_GRAD_NORM > 0:
        nn.utils.clip_grad_norm_(
            model.parameters(),
            MAX_GRAD_NORM
        )

    optimizer.step()

    log_probs = []
    values = []
    rewards = []
    dones = []
    entropies = []

    episode_rewards.append(episode_reward)

    if episode == 5:

```

```

        env.close()

    if episode % 10 == 0:
        avg_20 = np.mean(episode_rewards[-20:])
        remaining = EPISODES - episode

        print(
            f"[{episode}/{EPISODES}] "
            f"Faltam: {remaining} episódios | "
            f"Reward: {episode_reward:.2f} | "
            f"Média(20): {avg_20:.2f}"
        )

    env.close()
    plot_rewards(episode_rewards)

    elapsed = time.time() - start_time

    hours = int(elapsed // 3600)
    minutes = int((elapsed % 3600) // 60)
    seconds = int(elapsed % 60)

    print("\n=====")
    print("TREINAMENTO FINALIZADO")
    print("=====")
    print(f"Tempo total: {hours:02d}h {minutes:02d}m {seconds:02d}s")
    print("=====")

def plot_rewards(episode_rewards):
    rewards = np.array(episode_rewards)
    rolling_mean = []

    for i in range(len(rewards)):
        start = max(0, i - 19)
        rolling_mean.append(np.mean(rewards[start:i + 1]))

    plt.figure(figsize=(10, 5))
    plt.plot(episode_rewards, alpha=0.3, label="Recompensa por episódio")
    plt.plot(rolling_mean, label="Média móvel últimos 20 episódios")
    plt.xlabel("Episódio")
    plt.ylabel("Recompensa")
    plt.title("Treinamento A2C n-step no Simple Pong")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(PLOT_PATH)
    plt.show()

    print(f"Gráfico salvo em: {PLOT_PATH}")

```

```

if __name__ == "__main__":
    if N_STEPS < 1:
        raise ValueError("N_STEPS precisa ser maior ou igual a 1.")

    train()

```

B Código Completo do Ambiente Simple Pong

```

# simple_pong_env.py

import gymnasium as gym
from gymnasium import spaces
import numpy as np

class SimplePongEnv(gym.Env):
    metadata = {"render_modes": ["human"], "render_fps": 60}

    def __init__(self, render_mode=None):
        super().__init__()

        self.width = 800
        self.height = 400

        self.paddle_height = 70
        self.paddle_width = 10
        self.ball_size = 10

        self.agent_x = 30
        self.opponent_x = self.width - 40

        self.paddle_speed = 8
        self.ball_speed_x = 5
        self.ball_speed_y = 4

        self.render_mode = render_mode
        self.window = None
        self.clock = None

        # Ações do agente:
        # 0 = parado
        # 1 = subir
        # 2 = descer

        self.action_space = spaces.Discrete(3)

        # Estado:
        # [

```

```

#   ball_x,
#   ball_y,
#   ball_vx,
#   ball_vy,
#   agent_y,
#   opponent_y,
#   ball_y - agent_y
# ]

self.observation_space = spaces.Box(
    low=np.array([0, 0, -1, -1, 0, 0, -1], dtype=np.float32),
    high=np.array([1, 1, 1, 1, 1, 1, 1], dtype=np.float32),
    dtype=np.float32
)

self.reset()

def reset(self, seed=None, options=None):
    super().reset(seed=seed)

    self.ball_x = self.width / 2
    self.ball_y = self.height / 2

    direction = self.np_random.choice([-1, 1])
    self.ball_vx = direction * self.ball_speed_x
    self.ball_vy = self.np_random.uniform(-self.ball_speed_y,
    ↪ self.ball_speed_y)

    self.agent_y = self.height / 2 - self.paddle_height / 2
    self.opponent_y = self.height / 2 - self.paddle_height / 2

    obs = self._get_obs()
    info = {}

    return obs, info

def step(self, action):
    reward = 0
    terminated = False
    truncated = False

    # -----
    # Movimento do agente
    # -----
    if action == 1:
        self.agent_y -= self.paddle_speed
    elif action == 2:
        self.agent_y += self.paddle_speed

    self.agent_y = np.clip(
        self.agent_y,

```

```

        0,
        self.height - self.paddle_height
    )

    # -----
    # Movimento do oponente
    # -----
    self._move_opponent()

    # -----
    # Movimento da bola
    # -----
    self.ball_x += self.ball_vx
    self.ball_y += self.ball_vy

    # Colisão com topo e fundo
    if self.ball_y <= 0:
        self.ball_y = 0
        self.ball_vy *= -1

    if self.ball_y >= self.height - self.ball_size:
        self.ball_y = self.height - self.ball_size
        self.ball_vy *= -1

    # -----
    # Colisão com raquete do agente
    # -----
    if (
        self.ball_x <= self.agent_x + self.paddle_width
        and self.agent_y <= self.ball_y <= self.agent_y +
        ↪ self.paddle_height
    ):
        self.ball_x = self.agent_x + self.paddle_width
        self.ball_vx *= -1

        relative_hit = (self.ball_y - (self.agent_y +
        ↪ self.paddle_height / 2)) / (self.paddle_height / 2)
        self.ball_vy = relative_hit * 5

    # -----
    # Colisão com raquete do oponente
    # -----
    if (
        self.ball_x + self.ball_size >= self.opponent_x
        and self.opponent_y <= self.ball_y <= self.opponent_y +
        ↪ self.paddle_height
    ):
        self.ball_x = self.opponent_x - self.ball_size
        self.ball_vx *= -1

```

```

        relative_hit = (self.ball_y - (self.opponent_y +
        ↪ self.paddle_height / 2)) / (self.paddle_height / 2)
        self.ball_vy = relative_hit * 5

# -----
# Pontuação
# -----
if self.ball_x < 0:
    reward = -1
    terminated = True

if self.ball_x > self.width:
    reward = 1
    terminated = True

obs = self._get_obs()
info = {}

if self.render_mode == "human":
    self.render()

return obs, reward, terminated, truncated, info

def _move_opponent(self):
    """
    Oponente com inteligência média.

    Ele tenta seguir a bola, mas:
    - só reage quando a bola está indo em sua direção;
    - possui erro aleatório;
    - não se move rápido o suficiente para ser perfeito.
    """

    opponent_center = self.opponent_y + self.paddle_height / 2

    if self.ball_vx > 0:
        target_y = self.ball_y

        # erro proposital para não ser perfeito
        noise = self.np_random.normal(0, 25)
        target_y += noise

        if opponent_center < target_y - 10:
            self.opponent_y += self.paddle_speed * 0.75
        elif opponent_center > target_y + 10:
            self.opponent_y -= self.paddle_speed * 0.75

    else:
        # quando a bola vai para o agente, o oponente volta
        ↪ lentamente ao centro
        center_target = self.height / 2

```

```

        if opponent_center < center_target - 10:
            self.opponent_y += self.paddle_speed * 0.4
        elif opponent_center > center_target + 10:
            self.opponent_y -= self.paddle_speed * 0.4

self.opponent_y = np.clip(
    self.opponent_y,
    0,
    self.height - self.paddle_height
)

def _get_obs(self):
    return np.array([
        self.ball_x / self.width,
        self.ball_y / self.height,
        self.ball_vx / 10,
        self.ball_vy / 10,
        self.agent_y / self.height,
        self.opponent_y / self.height,
        (self.ball_y - self.agent_y) / self.height
    ], dtype=np.float32)

def render(self):
    import pygame

    if self.window is None:
        pygame.init()
        pygame.display.init()
        self.window = pygame.display.set_mode((self.width,
        ↪ self.height))
        pygame.display.set_caption("Simple Pong RL")

    if self.clock is None:
        self.clock = pygame.time.Clock()

    self.window.fill((0, 0, 0))

    pygame.draw.rect(
        self.window,
        (255, 255, 255),
        pygame.Rect(
            self.agent_x,
            self.agent_y,
            self.paddle_width,
            self.paddle_height
        )
    )

    pygame.draw.rect(
        self.window,

```

```

        (255, 255, 255),
        pygame.Rect(
            self.opponent_x,
            self.opponent_y,
            self.paddle_width,
            self.paddle_height
        )
    )

    pygame.draw.rect(
        self.window,
        (255, 255, 255),
        pygame.Rect(
            self.ball_x,
            self.ball_y,
            self.ball_size,
            self.ball_size
        )
    )

    pygame.display.flip()
    self.clock.tick(self.metadata["render_fps"])

def close(self):
    if self.window is not None:
        import pygame
        pygame.display.quit()
        pygame.quit()
        self.window = None
        self.clock = None

```